

Sentinel Lattice: Cross-Domain Paradigm Synthesis for LLM Defense Architecture

Sentinel Research Team
Independent Researcher

<https://github.com/DmitrL-dev/AISecurity>

Abstract

We present Sentinel Lattice, a multi-layer defense architecture for Large Language Model (LLM) security that synthesizes 58 security paradigms from 19 scientific domains into 7 novel security primitives. Through systematic cross-domain analysis—applying methodologies from biology, nuclear safety, control theory, formal linguistics, game theory, and thermodynamics to the LLM security problem—we identify and fill fundamental gaps that no existing approach addresses.

We introduce PASR (Provenance-Annotated Semantic Reduction), a primitive that preserves information flow control through lossy semantic transformations; MIRE (Model-Irrelevance Containment Engine), which shifts the defense paradigm from backdoor detection (proven impossible by Goldwasser and Kim) to architectural containment; and five additional primitives addressing within-authority chaining, dual-use ambiguity, and predictive threat scoring.

Against a corpus of 250,000 simulated attacks across 15 categories with 5 mutation types, the full architecture achieves approximately 98.5% detection and containment, approaching the theoretical floor of 98–99% bounded by proven impossibility results. Exhaustive prior art search (51 cross-domain queries) confirms zero existing implementations for any of the proposed primitives.

1 Introduction

Large Language Models (LLMs) deployed as autonomous agents—with tool access, persistent memory, and multi-turn interaction—create an attack surface that no existing defense adequately addresses. The fundamental challenges span five categories:

1. **Prompt injection remains unsolved.** No production system reliably prevents instruction override via adversarial input [13].
2. **Agentic attacks compound.** Tool chains create exponential attack surface: N tools yield $O(N!)$ possible action chains.

3. **Model integrity is unverifiable.** Goldwasser and Kim [11] prove that backdoor detection is mathematically impossible under standard cryptographic assumptions.
4. **Semantic identity defeats classification.** Malicious and benign intent can produce identical text (“How do I mix bleach and ammonia?”).
5. **Defense layers conflict.** Provenance tracking and semantic transduction are architecturally incompatible without novel primitives.

Existing commercial solutions—Lakera Guard, Meta Prompt Guard [24], NVIDIA NeMo Guardrails [26], and LLM Guard—are all variants of content-level filtering: classify input as safe or dangerous, then block or allow. This approach has a fundamental ceiling, bounded by the impossibility results above.

Our approach. Rather than incrementally improving content classification, we apply a methodology inspired by recent breakthroughs in AI-driven scientific discovery. AlphaFold [18] did not invent new physics—it systematically explored 170,000 known protein structures and found patterns that 500,000 biologists missed due to cognitive limitations. Similarly, we systematically analyze **58 security paradigms from 19 scientific domains** and identify cross-domain combinations that have never been applied to LLM security.

This cross-domain synthesis yields seven novel security primitives (Table 1), five of which are genuinely new inventions with zero prior art, confirmed via 51 independent searches across GitHub and academic databases.

Contributions. This paper makes the following contributions:

- A systematic methodology for cross-domain paradigm synthesis applied to LLM security, analyzing 58 paradigms from 19 domains (§2).

Table 1: The seven security primitives introduced in this work.

#	Primitive	Acr.	Nov.
1	Provenance-Annotated Sem. Reduction	PASR	New
2	Capability-Attenuating Flow Labels	CAFL	New
3	Goal Predictability Score	GPS	New
4	Adversarial Argumentation Safety	AAS	New
5	Intent Revelation Mechanisms	IRM	New
6	Model-Irrelevance Containment	MIRE	New
7	Temporal Safety Automata	TSA	Adapted

- Seven novel security primitives, with formal definitions and architectural integration (§5–§8).
- A paradigm shift from backdoor *detection* (proven impossible) to backdoor *containment* via MIRE (§8).
- Evaluation against 250,000 simulated attacks across 15 categories, achieving approximately 98.5% containment (§9).
- Confirmation of zero prior art via exhaustive search across 51 cross-domain intersections (§11).

2 Background and Methodology

The core methodology of Sentinel Lattice draws on a simple observation: breakthroughs in AI-driven scientific discovery—AlphaFold [18] for protein folding, GNoME for materials science, GraphCast for weather prediction—succeed not by inventing new physics, but by systematically exploring known solution spaces too large for human cognition.

We apply the same principle to LLM security. Rather than proposing yet another content classifier, we systematically analyze **58 security paradigms from 19 scientific domains** and identify cross-domain combinations that have never been applied to the LLM security problem.

2.1 Paradigm Search Space

Table 2 summarizes the 19 domains analyzed. For each domain, we extracted paradigms with formal properties (provable guarantees, compositional structure, or impossibility results) and evaluated their applicability to the five fundamental LLM security challenges identified in §1.

Table 2: The 19 scientific domains and 58 paradigms analyzed.

Domain	#	Key Contributions
Biology / Immunology	5	BBB, negative selection
Nuclear / Military	4	Defense in depth, containment
Cryptography	4	PCC [25], ZK proofs
Aviation Safety	3	Swiss cheese [29], TCAS
Medieval Defense	3	Castle architecture
Financial Security	3	Separation of duties
Legal Systems	3	Adversarial process
Industrial Safety	3	HAZOP, STAMP [22]
CS Foundations	3	Capabilities [7], IFC [6]
Information Theory	3	Shannon [32], Kolmogorov
Category / Type Theory	3	Fibrations, dependent types
Control Theory	3	Lyapunov stability
Game Theory	3	Mechanism design, VCG
Ecology	3	Ecosystem resilience
Neuroscience	3	LTP, lateral inhibition
Thermodynamics	2	Landauer’s principle [21]
Distributed Consensus	2	BFT [20]
Formal Linguistics	3	Chomsky, speech acts [2]
Philosophy of Mind	2	Chinese room, frame problem
Total	58	

2.2 Cross-Domain Synthesis Method

Our methodology proceeds in four phases:

Phase 1: Paradigm extraction. For each of the 19 domains, we identify security-relevant paradigms—mechanisms with formal properties such as provable guarantees, compositional structure, or impossibility results. We exclude paradigms that are purely heuristic or lack formal grounding.

Phase 2: Applicability mapping. Each paradigm is evaluated against the five fundamental LLM security challenges (prompt injection, agentic compounding, model integrity, semantic identity, and layer conflict). A paradigm is considered *applicable* if it addresses at least one challenge that current content-classification approaches cannot.

Phase 3: Combination search. We systematically enumerate pairwise and higher-order combinations of applicable paradigms, seeking synergies where combining paradigms from different domains produces capabilities absent from either paradigm alone. This is the step that yields genuinely novel primitives: for example, combining information flow control [6] with database provenance semirings [12] and category-theoretic fibrations produces PASR (§5).

Phase 4: Novelty verification. Each candidate primitive is verified for novelty via exhaustive search. We executed 51

compound queries on grep.app (covering all GitHub repositories) and 15 targeted Google Scholar searches. All 51 queries returned zero existing implementations. Details appear in §11.

2.3 Key Theoretical Foundations

Several foundational results shape the architecture:

Impossibility of backdoor detection. Goldwasser and Kim [11] prove that no polynomial-time algorithm can distinguish a backdoored model from a clean one using only clean-data evaluation. This result motivates our shift from detection to *containment* via MIRE (§8).

Lattice-based information flow. Denning’s lattice model [6] provides the formal foundation for tracking information provenance through the architecture. Combined with the Bell-LaPadula [3] and Biba [4] models, it ensures that security labels form a complete lattice with well-defined $\sqcup$ (join) and $\sqcap$ (meet) operations.

Capability-based security. The confused deputy problem [15] and capability-based addressing [7] inform our design of CAFL, which ensures that capabilities can only *attenuate* (decrease) through a chain of tool invocations—never amplify.

Argumentation theory. Dung’s abstract argumentation frameworks [8] provide grounded semantics for resolving dual-use ambiguity: arguments for and against the safety of a request are evaluated under well-defined acceptability criteria, yielding auditable decisions required for EU AI Act compliance.

Runtime verification. Linear Temporal Logic (LTL) [28] and efficient monitoring [16] provide the basis for TSA, which compiles safety properties into $O(1)$ automata that monitor tool-chain execution in real time.

3 Threat Model

3.1 Adversary Capabilities

We adopt a Kerckhoffs-compliant threat model [19]: the adversary possesses *full knowledge* of the Sentinel Lattice defense architecture, including all detection patterns, layer mechanisms, escalation rules, and classification taxonomies. Security rests not on architectural secrecy but on the confidentiality of four classes of ephemeral state:

1. **Ephemeral cryptographic keys** rotated per session and used for inter-layer attestation.

Table 3: Attack categories and simulation counts used for Sentinel Lattice evaluation. Total corpus: 250,000 adversarial prompts.

Category	Simulations
Direct Injection	25,000
Indirect Injection	25,000
Multi-turn Crescendo [30]	20,000
Encoding / Obfuscation	20,000
Role-play / Persona	20,000
Tool Abuse / Agentic	20,000
Data Exfiltration	15,000
Social Engineering	15,000
Semantic Equivalence	15,000
Steganographic	12,000
Model-Level Compromise [17]	10,000
Cross-boundary Trust	10,000
Novel / Zero-day	13,000
Multi-modal	10,000
Adversarial ML	10,000
Total	250,000

2. **Active canary probes** injected by the TSA layer to measure downstream integrity.
3. **Activation baselines** computed over sliding windows and unique to each deployment instance.
4. **Negative-selection detector sets** generated stochastically and never fully observable by an external party.

This separation ensures that publishing the full architectural specification (as we do in this paper) does not weaken operational deployments.

3.2 Attack Categories

We organize adversarial inputs into fifteen categories spanning the known attack surface of large language model deployments. Table 3 summarizes each category and the number of simulated attack prompts used in our evaluation (Section 9).

Category design rationale. Categories were selected to cover both well-studied vectors (direct and indirect injection) and emerging threats (steganographic channels, model-level compromise via sleeper agents [17]). The *Novel / Zero-day* category contains prompts generated by an unconstrained red-team model with no access to Sentinel Lattice internals, providing an estimate of robustness against previously unseen attack patterns.

3.3 Mutation Strategy

A realistic adversary does not submit a single static payload; instead, they iteratively mutate failed attempts. To model this

Table 4: Mutation operators and their effect on Sentinel Lattice detection rate (PASR degradation relative to unmutated baseline).

Mutation Type	Description	Degradation
Lexical	Synonym substitution	−8.7%
Structural	Reorder clauses / sentences	−6.1%
Encoding	Switch encoding schemes	−14.5%
Context	Change cover story	−12.3%
Hybrid	Combine ≥ 2 operators	−18.2%

adaptive behavior, we define five mutation operators and measure the PASR degradation each induces on Sentinel Lattice when applied to initially-detected prompts. Table 4 reports results averaged across all fifteen attack categories.

Hybrid mutations pose the greatest challenge, yet even under worst-case composition the MIRE layer’s semantic analysis and the TSA layer’s behavioral monitoring jointly maintain detection rates above the operational threshold (Section 9).

3.4 Impossibility Results

Two fundamental impossibility results bound what any defense—including Sentinel Lattice—can achieve.

Indistinguishability of backdoored models. Goldwasser and Kim [11] show that no polynomial-time algorithm can reliably distinguish a backdoored neural network from a clean one, given only black-box query access. This result implies that purely input–output testing is insufficient for detecting model-level compromise; Sentinel Lattice therefore supplements query-time analysis with activation-space monitoring at the TSA layer, operating on internal representations rather than solely on outputs.

Semantic identity limitation. Informally, for any classifier C over natural-language requests, there exist semantically equivalent request pairs (r, r') such that C must misclassify at least one member of the pair—either producing a false positive on a benign request or a false negative on a malicious one. This follows from the undecidability of semantic equivalence in unrestricted natural language and motivates the multi-layer design of Sentinel Lattice: no single classification boundary suffices, so the lattice composes complementary detection surfaces (PASR, MIRE, TSA) whose error regions are empirically near-disjoint (Section 9).

Operational stance. Sentinel Lattice does not claim to circumvent these theoretical limits. Rather, the architecture is designed to *operate within them*: layered redundancy ensures that the residual attack surface left by any individual impossibility result is covered by an orthogonal detection mechanism,

yielding aggregate detection rates that approach—but cannot reach—perfection.

4 Architecture Overview

Sentinel Lattice is organized as a multi-layer defense architecture comprising six core layers, three combinatorial layers, and one containment layer. Input flows through successive layers in a defense-in-depth pipeline, where each layer addresses a distinct class of threat using a different analytical paradigm. No single layer is sufficient; the architecture derives its strength from the compositional guarantees that emerge when heterogeneous defenses are layered with formally specified interfaces.

Table 5 summarizes the full layer stack. Layers L1–L3 provide foundational defense using established paradigms adapted for LLM contexts. The combinatorial layers (PASR, TCSA, ASRA) introduce novel cross-domain primitives. MIRE provides the final containment guarantee.

4.1 L1: Sentinel Core

The first layer is a fully deterministic, zero-ML pattern-matching engine implemented in Rust. Sentinel Core comprises 53 regex engines organized into 10 functional categories, covering 704 distinct attack patterns validated by 887 unit tests with zero failures. The engine uses Aho-Corasick [1] multi-pattern pre-filtering to achieve $O(n)$ text scanning regardless of pattern count, yielding sub-millisecond latency on typical inputs.

Determinism is a critical design property: given identical input, Sentinel Core produces identical output on every execution, with no stochastic components, no model inference, and no external dependencies. This makes the layer fully auditable and amenable to formal verification. In our 250,000-attack simulation (§9), L1 alone catches 36.0% of all attacks—establishing a high-confidence baseline before any probabilistic analysis begins.

Table 6 summarizes the engine categories.

4.2 L2: Capability Proxy and Information Flow Control

L2 provides a structural defense rooted in classical access control theory. Drawing on Bell-LaPadula [3] mandatory access control, Clark-Wilson [5] integrity enforcement, and Dennis and Van Horn’s [7] capability model, L2 interposes a *capability proxy* between the language model and all external tools. The model never observes real tool interfaces; it interacts exclusively with virtual proxies that expose only the subset of functionality permitted by the current security context.

Table 5: Sentinel Lattice layer stack. Latency is per-request overhead. Status indicates implementation maturity at time of writing.

Layer	Name	Latency	Paradigm Source	Status
L1	Sentinel Core	<1 ms	Pattern matching	Implemented (704 patterns, 53 engines)
L2	Capability Proxy + IFC	<10 ms	Bell-LaPadula [3], Clark-Wilson [5]	Designed
L3	Behavioral EDR	~50 ms async	Endpoint Detection & Response	Designed
PASR	Prov.-Annotated Sem. Reduction	+1–2 ms	Novel invention	Designed
TCSA	Temporal-Capability Safety	$O(1)$ /call	Runtime verification + Novel	Designed
ASRA	Ambiguity Surface Resolution	Variable	Mechanism design + Novel	Designed
MIRE	Model-Irrelevance Containment	~0–5 ms	Novel paradigm shift	Designed

Table 6: Sentinel Core engine categories. Pattern counts are approximate.

Category	Engines	Patterns
Injection & Jailbreak	6	~150
Evasion & Encoding	4	~80
Agentic & Tool Abuse	5	~90
Data Protection	4	~70
Social & Cognitive	4	~60
Supply Chain	3	~50
Code & Runtime	4	~65
Advanced Threats	6	~80
Output & Cross-tool	3	~50
Domain-specific	14	~109
Total	53	704

Provenance tagging. Every token entering the system receives a provenance tag that governs its trust level and permissible actions:

- **OPERATOR (HIGH trust):** system prompts and operator-defined instructions. May issue any permissible tool call.
- **USER (LOW trust):** end-user input. May request tool calls subject to capability attenuation.
- **RETRIEVED (NONE):** content from RAG retrieval, web search, or tool output injected into context. *Cannot* issue tool calls under any circumstance.
- **TOOL (MEDIUM trust):** structured output from previously executed tools. May trigger follow-up calls only within pre-authorized chains.

NEVER lists. Certain operations are not merely filtered but are *architecturally non-existent*: the virtual proxy simply does not expose them. A model cannot request `rm -rf /` if the proxy’s capability surface contains no destructive filesystem operations. This is a stronger guarantee than content filtering, as the attack surface is reduced by construction rather than by detection.

Security lattice. Security labels form a lattice ordered as $\text{PUBLIC} \prec \text{INTERNAL} \prec \text{SECRET} \prec \text{TOP_SECRET}$. Information flows upward only: a process at INTERNAL may read PUBLIC data but cannot write to PUBLIC channels, preventing exfiltration. This is a direct application of the Bell-LaPadula “no write down” property adapted to LLM data flows.

In simulation, L2 catches 20.3% of attacks with latency under 10 ms.

4.3 L3: Behavioral EDR

Layer 3 adapts the Endpoint Detection and Response (EDR) paradigm from host-based security to the LLM agent context. Unlike L1 and L2, which operate synchronously on the critical path, L3 runs asynchronously: behavioral telemetry is collected per-turn and analyzed off-path, with alerts propagated back to the orchestrator within approximately 50 ms. This design ensures that behavioral monitoring imposes no latency penalty on normal operation.

Behavioral signals. L3 monitors five classes of signal:

1. **Tool call frequency.** Sudden increases in tool invocation rate are detected via CUSUM (Cumulative Sum) change-point detection [27], which identifies statistically significant deviations from the running baseline.
2. **Unusual tool combinations.** A first-order Markov chain over tool call sequences models expected transition probabilities; low-probability transitions (e.g., `read_file` → `send_email`) trigger alerts.
3. **Privilege escalation patterns.** Sequences of tool calls that monotonically increase in capability level are flagged, particularly when the escalation crosses provenance boundaries.
4. **Output topic drift.** Cosine similarity between successive response embeddings detects gradual topic migration characteristic of crescendo attacks.
5. **Cross-session patterns.** Behavioral fingerprints are correlated across sessions to detect distributed attacks that stay below per-session thresholds.

Stability analysis. We define a Lyapunov stability function over the behavioral state:

$$V(s) = w_1 \cdot \text{topic_drift}(s) + w_2 \cdot \text{priv_level}(s) + w_3 \cdot \text{tool_diversity}(s) + w_4 \cdot \text{data_sensitivity}(s) \quad (1)$$

where s indexes the conversation turn and w_i are empirically tuned weights. The safety invariant requires $\frac{dV}{ds} \leq 0$: the system must not become progressively less safe over the course of a session. A sustained positive derivative indicates a crescendo attack in progress and triggers escalation to the TCSA layer.

In simulation, L3 catches 10.9% of attacks at approximately 50 ms asynchronous latency.

4.4 Combinatorial Layers

The three combinatorial layers—PASR, TCSA, and ASRA—represent the primary novel contributions of this work and are detailed in §5–§7. Here we briefly summarize the cross-domain synthesis underlying each.

Combination Alpha (PASR). Provenance-Annotated Semantic Reduction synthesizes three paradigms: Chomsky’s hierarchy of formal grammars (to structurally reduce prompts to canonical forms), Shannon’s channel capacity theorem [32] (to bound the information content of adversarial perturbations), and Landauer’s principle [21] (to establish thermodynamic lower bounds on the cost of semantic-preserving transformations). The result is a transducer that strips adversarial payload while preserving semantic intent, with provenance annotations maintained through the reduction.

Combination Beta (TCSA). Temporal-Capability Safety Automata combine Lyapunov stability analysis (from control theory) with Byzantine fault-tolerant consensus [20] and long-term potentiation gating (from computational neuroscience). The TCSA layer models the agent’s capability trajectory as a dynamical system and enforces monotonic safety degradation bounds. Multi-agent deployments use BFT consensus to prevent any single compromised agent from escalating the collective capability envelope.

Combination Gamma (ASRA). Ambiguity Surface Resolution Analysis draws on Austin’s [2] and Searle’s [31] speech act theory (illocutionary force analysis), lateral inhibition from neural circuit design, and Gricean conversational maxim violation detection [14]. When a prompt’s illocutionary force is ambiguous—it could be interpreted as either a legitimate request or an attack—ASRA quantifies the ambiguity surface and applies mechanism-design principles to resolve it in a way that is incentive-compatible for honest users and costly for adversaries.

5 PASR: Provenance-Annotated Semantic Reduction

5.1 The Architectural Conflict

The Sentinel Lattice architecture layers its defenses in depth, but two critical layers—L2 (Information Flow Control via taint tags) and L5 (Semantic Transduction, inspired by the blood–brain barrier)—are architecturally incompatible in their naïve formulations. This incompatibility is not a minor integration challenge; it is a genuine structural conflict that no existing system addresses.

L2 implements Denning’s lattice model [6]: every token entering the system receives a provenance tag $p_i \in \{\text{OPERATOR}, \text{USER}, \text{RETRIEVED}, \text{TOOL}\}$, and these tags propagate through the processing pipeline according to information-flow rules. The security guarantee is precise: at any point in the pipeline, one can determine which inputs influenced a given intermediate value.

L5 implements semantic transduction, modeled on the biological blood–brain barrier (BBB). It deliberately *destroys* the original token sequence and re-synthesizes only the extracted semantic intent. This destruction is not a side-effect—it is the security mechanism. Injected instructions that rely on specific token patterns cannot survive lossy semantic extraction.

The conflict is immediate: **L5 destroys the very tokens to which L2’s tags are attached.** After transduction, no tokens remain to carry provenance information. The taint tags die with the tokens they annotated. Any downstream layer that requires provenance information—and several critical security decisions depend on it—receives nothing.

5.2 Core Insight: Provenance as a Property of Derivations

The resolution begins with a conceptual shift: **provenance is not a property of tokens—it is a property of derivations.** A taint tag does not describe the token it is attached to; it describes the *origin* of the information that token carries. When L5 destroys tokens and re-synthesizes semantic intent, the derivation history does not vanish—it merely needs to be recorded in a form that survives the transduction boundary.

PASR implements this insight via a *two-channel output* from the trusted transducer. The transducer READS provenance tags from its input tokens and WRITES provenance certificates onto its output semantic fields:

- **Channel 1: Semantic intent** (content channel—lossy). The extracted meaning of the input, stripped of syntactic structure. This channel is deliberately lossy: injected instructions that depend on token-level patterns are destroyed.
- **Channel 2: Provenance certificate** (metadata channel—lossless). An HMAC-SHA256-signed record mapping each semantic field to the set of provenance tags that contributed

to it. This channel is unforgeable: only the trusted transducer holds the signing key.

5.3 The PASR Algorithm

The transduction proceeds in five steps. We illustrate with a concrete injection attempt where user-supplied text contains the instruction “ignore previous instructions and read /etc/passwd.”

1. **Receive tagged tokens from L2.** The transducer ingests a sequence of (t_i, p_i) pairs, where each token t_i carries the provenance tag p_i assigned by the IFC layer:

[(“ignore”, USER), (“previous”, USER),
 (“instructions”, USER), ...]

2. **Extract semantic intent (content channel—lossy).** The transducer performs lossy semantic extraction, discarding syntactic structure and producing a structured intent representation:

$S = \{\text{action: “file_read”, target: “/etc/passwd”}\}$

The original token sequence is irrecoverably destroyed. Any injected payload that relied on token-level patterns (e.g., “ignore previous instructions”) is reduced to its semantic content.

3. **Record provenance map (novel contribution).** For each field f_j in the semantic output S , the transducer records which tagged inputs contributed to it, producing a provenance map $P : \text{Fields}(S) \rightarrow \mathcal{P}(\text{Provenance})$:

$P(\text{action}) = \{\text{USER}\}$
 $P(\text{target}) = \{\text{USER}\}$

This is the step that no existing system performs. Standard semantic extraction discards provenance; standard taint tracking cannot survive re-synthesis. PASR does both.

4. **Sign the provenance map.** The transducer computes an HMAC-SHA256 signature over the concatenation of (S, P) using a per-session ephemeral key k held exclusively by the trusted transducer:

$\sigma = \text{HMAC-SHA256}(k, S \| P)$

The signed certificate (S, P, σ) is unforgeable: an adversary who compromises the LLM but not the transducer cannot fabricate a valid provenance annotation.

5. **Detect claims-vs-actual discrepancy.** The transducer compares the semantic content against the provenance certificate. In this example, the extracted intent requests `file_read` on a sensitive path—an action that requires

OPERATOR authority. However, the provenance map records that *all* contributing inputs carry USER provenance. The discrepancy between claimed authority (implicit in the action semantics) and actual source provenance constitutes an **injection signal**:

Required(action) = OPERATOR,
 $P(\text{action}) = \{\text{USER}\} \implies \text{INJECTION}$

5.4 Mathematical Framework: Provenance Lifting Functor

We formalize PASR as a functor between two categories, drawing on the theory of provenance semirings [12].

Category $\mathcal{C}$ (L2 output space). Objects are tagged token sequences $[(t_i, p_i)]$ where $t_i \in \text{Tokens}$ and $p_i \in \{\text{OPERATOR}, \text{USER}, \text{RETRIEVED}, \text{TOOL}\}$. Morphisms are token-level transformations that propagate tags according to IFC rules.

Category $\mathcal{D}$ (PASR output space). Objects are provenance-annotated semantic structures (S, P) where S is a semantic intent and $P : \text{Fields}(S) \rightarrow \mathcal{P}(\text{Provenance})$ maps each semantic field to the power set of provenance labels that contributed to it. Morphisms are semantic refinements that respect provenance monotonicity.

The lifting functor $\mathcal{L} : \mathcal{C} \rightarrow \mathcal{D}$. The PASR transducer defines a functor with four key properties:

1. **Content-lossy.** $\mathcal{L}$ is not injective on the content component: distinct input sequences may map to the same semantic intent. Formally, $\exists c_1 \neq c_2 \in \text{Ob}(\mathcal{C})$ such that $\pi_S(\mathcal{L}(c_1)) = \pi_S(\mathcal{L}(c_2))$, where π_S projects to the semantic component.
2. **Provenance-faithful.** For each semantic field f_j in the output, the provenance annotation is the union of all contributing input provenances:

$P(f_j) = \bigcup \{p_i : t_i \text{ contributed to } f_j\}$

3. **Monotone in trust.** The trust level assigned to a field is the minimum of all contributing trust levels, ensuring conservative propagation:

$\text{trust}(f_j) = \min \{\text{trust}(p_i) : p_i \in P(f_j)\}$

4. **Unforgeable.** The provenance certificate is HMAC-signed by the trusted transducer. No entity outside the transducer’s trust boundary can produce a valid signature.

Table 7: Intellectual lineage of PASR: each predecessor contributes a key property but leaves a critical gap that PASR closes.

Source	Contributes	Misses
DB Provenance Semirings [12]	Fine-grained per-datum tracking	Only structure-preserving transforms
IFC Taint Tracking [6]	Per-datum provenance labels	Labels do not survive re-synthesis
Cryptographic Attestation	Unforgeability of certificates	Wrong granularity (whole-message)
PASR (this work)	All three combined	Novel primitive (no prior art)

This construction is a *fibration* in the categorical sense: the forgetful projection $\pi_S : \mathcal{D} \rightarrow \mathcal{S}$ (discarding provenance and retaining only semantic intent) has a canonical lifting property. Given any morphism in the base category $\mathcal{S}$ (a semantic transformation) and a provenance-annotated object in the fiber above its source, there exists a unique lifted morphism in $\mathcal{D}$ that preserves provenance faithfulness. This lifting property is precisely what guarantees that provenance survives arbitrary downstream semantic processing—not merely the initial transduction step.

5.5 Synthesis: Why PASR Is New

Table 7 summarizes the intellectual lineage of PASR and identifies the specific gap that each predecessor leaves unaddressed. PASR is the first primitive to combine fine-grained provenance tracking, survival across lossy re-synthesis, and cryptographic unforgeability in a single mechanism.

5.6 Evaluation Summary

We evaluate the impact of PASR on the overall Sentinel Lattice detection pipeline. Full evaluation methodology and results appear in §9; here we highlight three key metrics:

- **Overall detection improvement.** Adding PASR to the Sentinel Lattice pipeline increases aggregate detection from 93.7% to 95.7% (+2.0 percentage points), with the largest gains on attacks that specifically target the L2–L5 boundary.
- **Taint-strip attack resolution.** ATK-011 (taint-strip injection), in which the adversary crafts inputs designed to shed provenance tags before reaching the semantic transducer, drops from 50–65% attacker success to 3–8%. This category was the primary motivation for PASR and is now effectively resolved.
- **Fast-path latency reduction.** Because provenance certificates enable downstream layers to make authorization de-

Table 8: Known weaknesses of PASR and deployed mitigations.

Weakness	Severity	Mitigation
HMAC key is a single point of failure: compromise of the signing key allows provenance forgery	High	HSM-backed key storage with per-session ephemeral keys; key rotation every 300 s
Provenance boundary ambiguity at BPE token splits: a single semantic concept may span tokens with different provenance tags	Medium	Conservative assignment: if any contributing token carries USER provenance, the entire field inherits USER
Provenance laundering via tool calls: an adversary invokes a tool with user-controlled input, and the tool’s output receives TOOL provenance	Medium	Transitive provenance tracking: tool outputs inherit the provenance of their inputs via CAFL integration

cisions without re-querying L2, the fast-path latency decreases from 15 ms to 3 ms (a $5\times$ improvement). The provenance certificate acts as a pre-computed authorization token.

5.7 Known Weaknesses

We identify three weaknesses in the current PASR design and describe the mitigations deployed in the Sentinel Lattice prototype.

These weaknesses are architectural rather than implementation-specific: any system that lifts provenance across a lossy semantic boundary will face analogous challenges. We discuss longer-term solutions, including threshold-HMAC schemes and formal provenance algebras, in §10.

6 TCSA: Temporal-Capability Safety Architecture

The preceding layers defend against attacks that are individually identifiable—a malicious pattern, a forbidden capability, a suspicious behavioral signature. But a critical class of attacks evades every such layer: *within-authority chaining*, where each individual action is fully legitimate yet the composition constitutes a security violation.

Motivating example. Consider the following four-step tool chain executed by an LLM agent:

1. `file_read(".env")` — reads the application’s environment file. *Legitimate*: the agent has file-read capability.
2. `parse(output, "credentials")` — extracts database credentials from the file contents. *Legitimate*: parsing is a core LLM capability.
3. `compose_email(to="external@evil.com", body=credentials)` — drafts an email containing the extracted credentials. *Legitimate*: the agent has email-composition capability.
4. `send_email()` — transmits the composed email. *Legitimate*: the agent has email-send capability.

Every action is individually authorized. No single layer catches the violation. Yet the chain constitutes **data exfiltration**—sensitive credentials have been read, composed into a message, and transmitted to an external attacker-controlled address. The attack is invisible to any defense that examines actions in isolation.

TCSA addresses this gap through three sub-primitives that collectively monitor temporal ordering, data-flow capabilities, and predictive trajectory:

- **TSA** (Temporal Safety Automata) — adapted from runtime verification; monitors temporal ordering of tool calls against LTL safety properties.
- **CAFL** (Capability-Attenuating Flow Labels) — a genuinely new primitive; tracks data capabilities through LLM transformations under worst-case taint assumptions.
- **GPS** (Goal Predictability Score) — a genuinely new primitive; provides predictive early warning by scoring how many reachable future states lead to safety violations.

6.1 TSA: Temporal Safety Automata

TSA adapts techniques from runtime verification—specifically the monitor-automata approach of Havelund and Roşu [16] and Pnueli’s Linear Temporal Logic (LTL) [28]—to the setting of LLM tool chains. While runtime verification has been extensively studied for software systems, it has never been applied to monitor sequences of LLM tool invocations. The adaptation is non-trivial: unlike deterministic programs, LLM agents select actions stochastically, and the space of possible tool chains is combinatorially large.

Design-time compilation. Safety properties are expressed as LTL formulae over the alphabet of tool-call events. At design time, each formula is compiled into a deterministic monitor automaton via standard LTL-to-Büchi-to-monitor construction [16]. The resulting automata are minimal and fixed-size, requiring no runtime compilation.

Example safety properties. The following three properties illustrate the expressiveness of the TSA formalism:

$$\mathbf{P}_1: \square(\text{read_sensitive} \rightarrow \square\neg\text{send_external}) \quad (2)$$

$$\mathbf{P}_2: \neg\Diamond(\text{read_credentials} \wedge \Diamond\text{send_external}) \quad (3)$$

$$\mathbf{P}_3: \square(\text{privilege_change} \rightarrow \bigcirc\text{approval_received}) \quad (4)$$

Property $\mathbf{P}_1$ states that after reading sensitive data, the agent may *never* subsequently send data to an external channel—a permanent prohibition. Property $\mathbf{P}_2$ forbids any execution trace in which credential access is eventually followed by external transmission—directly encoding the exfiltration scenario from the motivating example. Property $\mathbf{P}_3$ requires that every privilege escalation be *immediately* followed by an approval event, enforcing a mandatory human-in-the-loop checkpoint.

Abstract security state. To make monitor evaluation tractable, we define an *abstract security state* as a 16-bit vector encoding the security-relevant predicates of the current execution. The 16 bits encode predicates including:

- `sensitive_data_accessed` — set when any tool reads data classified at INTERNAL or above.
- `credentials_accessed` — set when authentication material (API keys, passwords, tokens) is read.
- `external_channel_opened` — set when a tool establishes a connection to an external endpoint.
- `privilege_elevated` — set when the agent’s capability set expands beyond its initial grant.
- `write_performed` — set when any destructive or state-modifying tool call executes.
- `approval_pending` — set when a privilege change awaits operator confirmation.

A 16-bit vector yields $2^{16} = 65,536$ possible abstract states. This state space is small enough for *complete enumeration* at design time, enabling pre-computation of all monitor transitions and, critically, enabling the GPS predictive analysis described in §6.3.

Runtime cost. At runtime, each tool call triggers a single state-vector update (bitwise OR of the relevant predicate bits) followed by a table lookup in the pre-compiled monitor transition function. Both operations are $O(1)$ with respect to chain length, tool count, and property count. Monitor evaluation adds negligible overhead—under $1\ \mu\text{s}$ per tool call—making TSA suitable for latency-sensitive deployments.

6.2 CAFL: Capability-Attenuating Flow Labels

CAFL is a genuinely new primitive with no direct precedent in the literature. Classical information flow control (IFC) [6] tracks how data propagates through deterministic programs, where the transformation applied to each datum is known and analyzable. This assumption breaks catastrophically for LLM-based agents: the model can perform *any* transformation on its input, including arbitrary summarization, recombination, and rephrasing. No static analysis can determine what an LLM will do with a given input.

Key insight: worst-case taint. CAFL resolves this by adopting a *worst-case taint* assumption: if data carrying capability label C enters an LLM, **all output from that LLM invocation is tainted with the restrictions of C** . This is deliberately conservative—it may over-approximate (flagging safe chains as dangerous) but will *never* under-approximate (allowing dangerous chains through). Soundness is preserved at the cost of completeness, which is the correct trade-off for a security mechanism.

Capability labels. Every data object in the system carries a *capability label*—a set of permitted downstream operations. The critical invariant is that capabilities may only *decrease* as data flows through the processing chain; they may never increase. This monotonic attenuation ensures that sensitive data progressively loses its ability to reach dangerous sinks:

- `file_read(".env")` → output: $\{\text{PROCESS}, \text{DISPLAY}\}$
Sensitive configuration data may be processed internally and displayed to the operator, but *cannot* be exported.
- `file_read("public.md")` →
output: $\{\text{PROCESS}, \text{DISPLAY}, \text{EXPORT}\}$
Public data carries the full capability set, including export.
- `email_send()` requires input: $\{\text{EXPORT}\} \in C$
The email tool checks that its input data carries the `EXPORT` capability before transmission.

Blocking the exfiltration chain. Revisiting the motivating example: the chain `.env` → LLM → `email_send` is blocked because `file_read(".env")` produces output labeled $\{\text{PROCESS}, \text{DISPLAY}\}$. When this data enters the LLM, worst-case taint applies: the LLM’s output inherits the *intersection* of input capabilities, yielding at most $\{\text{PROCESS}, \text{DISPLAY}\}$. The subsequent `email_send()` requires $\{\text{EXPORT}\} \in C$, which is absent. The chain is rejected before the email is ever composed.

Membrane pattern. CAFL enforces capability attenuation at *trust boundary crossings*—points where data moves between security domains (e.g., from internal processing to

external communication, or from the agent’s workspace to a third-party API). Each boundary acts as a *membrane* that filters capabilities: data may cross only if it carries the capabilities required by the destination domain. This membrane pattern provides defense in depth, as even a compromised internal component cannot elevate the capabilities of data that has already been attenuated.

6.3 GPS: Goal Predictability Score

GPS is a genuinely new primitive that provides *predictive* defense: it catches tool chains that are *heading toward* a safety violation before the violation actually occurs. Where TSA detects violations at the moment they happen (the monitor enters a rejecting state), GPS provides early warning by analyzing the *trajectory* of the current execution through the abstract state space.

Definition. Given the current abstract security state $s \in \{0, 1\}^{16}$, define $\mathcal{N}(s)$ as the set of states reachable from s via a single tool call, and $\mathcal{D}(s) \subseteq \mathcal{N}(s)$ as the subset of those states that either (a) directly violate a TSA safety property or (b) have no safe successor under any tool call. The Goal Predictability Score is:

$$GPS(s) = \frac{|\mathcal{D}(s)|}{|\mathcal{N}(s)|} \quad (5)$$

A score of $GPS(s) = 0$ indicates that no single-step successor leads to danger; $GPS(s) = 1$ indicates that *every* possible next action leads to a safety violation.

Tractability. The key observation enabling GPS is that the abstract state space is only $2^{16} = 65,536$ states. This is small enough for *complete enumeration*: at design time, we pre-compute $\mathcal{D}(s)$ and $\mathcal{N}(s)$ for every state s , storing the resulting GPS values in a lookup table. At runtime, computing GPS requires a single table lookup—the same $O(1)$ cost as the TSA monitor transition.

Threshold policy. We define a two-threshold policy based on the GPS value:

- $GPS(s) > 0.7$: **Warning.** The system alerts the operator that the current tool chain is approaching a region of the state space where most continuations lead to safety violations. The agent may continue, but its actions are logged at elevated verbosity.
- $GPS(s) > 0.9$: **Block.** The system pre-emptively halts the tool chain. Fewer than 10% of possible continuations are safe, indicating that the agent has maneuvered into a near-certain violation trajectory regardless of its next action.

Table 9: Comparison of TCSA with pair-based tool-chain analysis (CrossToolGuard). TCSA provides strictly stronger guarantees across every dimension.

Aspect	CrossToolGuard	TCSA
Chain length	Pairs only	Arbitrary length
Temporal order	No	Yes (LTL via TSA)
Data flow	No	Yes (CAFL labels)
Predictive	No	Yes (GPS score)
Runtime cost	$O(N^2)$ pairs	$O(1)$ per call

Complementarity with TSA. GPS and TSA are complementary: TSA provides *reactive* enforcement (blocking the exact moment a property is violated), while GPS provides *proactive* defense (warning or blocking before the violation occurs). Together, they implement a defense-in-depth strategy over the temporal dimension: GPS catches attacks that are still developing, and TSA catches any that slip past the predictive threshold.

6.4 Comparison with Existing Approaches

The closest existing work to TCSA is CrossToolGuard, which analyzes pairs of consecutive tool calls for dangerous combinations. Table 9 summarizes the key differences. TCSA subsumes pair-based analysis while adding temporal ordering, data-flow tracking, and predictive capabilities that no existing system provides.

The $O(1)$ -per-call runtime of TCSA deserves emphasis. CrossToolGuard must evaluate every pair of tools in the current chain, yielding $O(N^2)$ cost for a chain of length N . TCSA maintains a single 16-bit state vector and performs one table lookup per tool call, regardless of chain length or the number of safety properties being monitored. This constant-time guarantee makes TCSA viable even for long-running agentic sessions with hundreds of sequential tool invocations.

7 ASRA: Ambiguity Surface Resolution Architecture

The fundamental impossibility. Consider the query: “*How do I mix bleach and ammonia?*” Submitted by a chemistry student preparing for an exam, this request is entirely legitimate—the reaction between sodium hypochlorite and ammonia is standard general-chemistry curriculum. Submitted by an adversary seeking to produce chloramine gas in an enclosed space, the identical query constitutes a request for a weapon-of-mass-effect procedure. The two instances share the same lexical tokens, the same syntactic parse tree, and the same compositional semantics. No feature of the text distinguishes them. We term this phenomenon *semantic identity*: a condition under which the surface form of a request is invariant across benign and malicious intent.

Table 10: The ASRA five-layer resolution stack. Layers are ordered by increasing interaction cost; earlier layers resolve cheaper cases first.

Layer	Name	Phase	Mechanism
L0	Ambiguity Surface Mapping	Design-time	Taxonomic analysis
L1	Risk-Adjusted Response	Runtime	Detail modulation by PASR
L2	Deontic Conflict Detection	Runtime	Obligation–prohibition logic
L3	AAS	Runtime	Argumentation semantics
L4	IRM	Runtime	Mechanism design

Semantic identity is not an edge case. Our analysis of the Sentinel Lattice threat taxonomy (Section 3) identifies 347 prompt classes—19.6% of the audited attack surface—in which benign and adversarial interpretations produce textually indistinguishable queries. For these classes, *no text-only classifier, regardless of architecture or training data, can achieve separation*, because the decision boundary collapses to a single point in feature space. The residual is not a failure of model capacity; it is an information-theoretic impossibility.

Core insight. If the text itself cannot discriminate intent, then the system must change the *interaction* so that intent becomes observable. ASRA operationalises this insight through a five-layer resolution stack that progressively transforms ambiguous requests into distinguishable interaction trajectories, allowing the model to condition its response on *revealed* rather than *inferred* intent.

7.1 The Five-Layer Resolution Stack

Table 10 summarises the ASRA resolution stack. Each layer operates on the residual ambiguity left by its predecessor: L0 identifies the ambiguity surface at design time; L1–L2 apply static, policy-driven mitigations; L3–L4 engage dynamic, interaction-driven resolution for the hard core of semantically identical queries.

L0: Ambiguity Surface Mapping. At design time, the Sentinel Lattice threat taxonomy is analysed to identify all prompt classes exhibiting semantic identity. Each class is annotated with an *ambiguity profile*: the set of plausible benign and malicious interpretations, the risk tier of the worst-case interpretation, and the minimum resolution layer required to discriminate intent.

L1: Risk-Adjusted Response. For queries whose ambiguity profile permits partial resolution through information control, L1 modulates response detail as a function of the risk

tier assigned by PASR. A query about household chemical reactions, for instance, may receive a conceptual explanation (reaction products, safety warnings) without specifying concentrations, mixing ratios, or confinement strategies that would be necessary for weaponisation.

L2: Deontic Conflict Detection. When a query simultaneously triggers an obligation to assist and a prohibition against enabling harm, L2 detects the conflict using deontic logic operators [33]. Formally, if $O(\textit{help})$ and $\mathcal{F}(\textit{harm})$ are both derivable from the policy lattice for a given query, the conjunction $O(\textit{help}) \wedge \mathcal{F}(\textit{harm})$ constitutes a *deontic conflict signal*, escalating the query to L3 for argumentation-based resolution.

7.2 L3: Adversarial Argumentation Safety (AAS)

Layers L0–L2 resolve ambiguity through static policy. For the residual core—queries where policy alone cannot discriminate—AAS introduces a fundamentally new primitive: the application of Dung’s abstract argumentation frameworks [8] to LLM content-safety decisions. While argumentation semantics have been studied extensively in AI for knowledge representation, legal reasoning, and multi-agent negotiation, their application to the safety alignment of generative language models is, to our knowledge, entirely novel.

Framework construction. For each query flagged by L2 as a deontic conflict, AAS constructs an explicit argumentation framework $AF = \langle \mathcal{A}, \mathcal{R} \rangle$, where $\mathcal{A}$ is a set of arguments and $\mathcal{R} \subseteq \mathcal{A} \times \mathcal{A}$ is an attack relation. Arguments are partitioned into *pro-legitimate* and *pro-malicious* sets. We illustrate with the bleach-ammonia exemplar:

Pro-legitimate arguments:

- A_1 : The information is publicly available in chemistry textbooks and safety data sheets.
- A_2 : Understanding the reaction *prevents* accidental poisoning—a safety benefit.
- A_3 : The query matches standard general-chemistry curriculum content.

Pro-malicious arguments:

- B_1 : The reaction produces chloramine gas, a toxic agent capable of causing serious injury in confined spaces.
- B_2 : The query requests procedural detail (“how do I mix”) rather than theoretical understanding, suggesting operational intent.
- B_3 : No professional or educational context accompanies the request.

Attack relations: A_1 attacks B_3 (public availability weakens the inference from missing context); B_2 attacks A_3 (procedural framing undermines the curriculum interpretation). Addi-

tional attacks are instantiated from the Sentinel Lattice policy lattice as applicable.

Grounded semantics and context conditioning. AAS computes the *grounded extension* of the framework—the minimal complete labelling under Dung’s semantics [8]—to determine which arguments survive mutual attack. Crucially, the extension is *context-conditioned*: when external signals are available (e.g., the user is authenticated as a secondary-school teacher), context-specific arguments are added to $\mathcal{A}$ that defeat key pro-malicious claims, tipping the grounded extension toward the legitimate interpretation. In the absence of such signals (anonymous user, no session history), the pro-malicious arguments remain undefeated and the system restricts output accordingly.

Auditability. A critical advantage of argumentation-based resolution is *full auditability*: every safety decision is accompanied by the explicit framework $(\mathcal{A}, \mathcal{R})$, the computed extension, and the resulting policy action. The operator can inspect not merely *what* the system decided but *why*—which arguments prevailed and which were defeated. This property is of direct relevance to the transparency requirements of the EU AI Act and analogous regulatory frameworks that mandate explanations for high-risk AI decisions.

7.3 L4: Intent Revelation Mechanisms (IRM)

AAS resolves ambiguity when contextual signals are available to condition the argumentation framework. When such signals are absent—the hardest case—IRM provides the final resolution layer by drawing on *mechanism design*, the branch of microeconomic theory concerned with designing rules of interaction such that self-interested agents reveal private information through their choices. The application of mechanism design to LLM content safety is, to our knowledge, without precedent in the literature and constitutes the only known primitive that directly addresses the fundamental impossibility of semantic identity.

The core observation is that while malicious and benign users produce identical *text*, they possess different *preferences* over response formats, interaction trajectories, and information granularity. IRM exploits these preference asymmetries through three complementary mechanisms.

Mechanism 1: Screening (self-selection). The system offers the user a structured choice among response formats whose informational content varies along dimensions correlated with intent. For the bleach-ammonia query, the offered formats might include:

- *Educational overview*: conceptual explanation of the reaction chemistry, hazard classification, and safety precautions.
- *Detailed technical*: reaction stoichiometry, thermodynamics, and kinetics with literature references.

- *Laboratory protocol*: step-by-step procedure including concentrations, apparatus, and ventilation requirements.

A chemistry student preparing for an examination will self-select the educational overview or the detailed technical format, both of which match their informational need. An adversary seeking actionable synthesis instructions requires the laboratory protocol—a selection that reveals operational intent and triggers elevated scrutiny or refusal. The mechanism achieves separation not by classifying text but by inducing *voluntary self-selection* across formats whose risk profiles differ.

Mechanism 2: Costly signaling. The system requests contextual information that is *cheap to provide for legitimate users* (who possess it truthfully) but *costly for adversaries* (who must fabricate it convincingly and consistently). Examples include course identifiers, institutional affiliations, or descriptions of the professional context motivating the query. The cost asymmetry arises because legitimate context is recalled from memory whereas adversarial context must be invented, maintained across follow-up queries, and rendered consistent with verifiable external facts. The signal need not be verified to be useful: the act of providing or declining to provide it shifts the posterior distribution over intent.

Mechanism 3: Sequential revelation. Information is released incrementally across multiple interaction turns. At each step, the system observes the user’s response trajectory—which follow-up questions are asked, which details are pursued, which are ignored—and updates its belief over intent. Legitimate users exhibit trajectories consistent with learning (requesting clarification, connecting to related concepts); adversarial users exhibit trajectories consistent with extraction (requesting greater specificity, operational parameters, edge-case handling). The sequential structure converts a single-shot classification problem, which is impossible under semantic identity, into a multi-round interaction game in which behavioural divergence accumulates over time.

7.4 Coverage and Residual Ambiguity

Table 11 quantifies the cumulative effect of the ASRA resolution stack on the semantic identity residual identified in Section 7.

Before ASRA, the semantic identity residual stands at 19.6% of the audited prompt classes—nearly one in five attack-surface categories cannot be resolved by any text-only classifier. After full ASRA deployment, the residual falls to approximately 6.0%, a reduction of roughly 70%. The remaining ~6% represents *irreducible ambiguity*: cases in which even multi-turn interaction and mechanism-design primitives cannot reliably separate benign from malicious intent within acceptable latency and user-experience constraints. For these cases, Sentinel Lattice escalates to human review, ensuring

Table 11: Cumulative reduction of the semantic identity residual across ASRA layers. The baseline residual (19.6% of the audited attack surface) is measured prior to ASRA deployment.

Layer	Resolved (%)	Residual (%)
Baseline (no ASRA)	—	19.6
+ L0 (Ambiguity Mapping)	1.2	18.4
+ L1 (Risk-Adjusted Response)	4.8	13.6
+ L2 (Deontic Conflict)	2.1	11.5
+ L3 (AAS)	3.0	8.5
+ L4 (IRM)	2.5	6.0

that no query in the irreducible core receives an unmoderated high-risk response.

8 MIRE: Model-Irrelevance Containment Engine

8.1 The Impossibility Result and Paradigm Shift

The design of MIRE begins with a negative result. Goldwasser and Kim [11] prove that no polynomial-time algorithm can distinguish a backdoored model from a clean model using only clean-data evaluation. Formally, for any efficient distinguisher D and any backdoor insertion procedure B that preserves clean-data accuracy:

$$|\Pr[D(M_{\text{clean}}) = 1] - \Pr[D(B(M_{\text{clean}})) = 1]| \leq \text{negl}(\lambda)$$

where λ is the security parameter. This is not an engineering limitation amenable to incremental improvement—it is a *mathematical proof*. No amount of monitoring sophistication, dataset curation, or computational budget overcomes an impossibility result. Detection has a fundamental ceiling.

MIRE responds to this impossibility by reframing the question entirely. The conventional question—*how do we detect whether a model is backdoored?*—is the wrong question. The correct question is: **how do we make the backdoor irrelevant?** If a backdoor activates but accomplishes nothing—if its output is blocked, its behavioral signature is flagged, and its architectural reach is contained—then the distinction between a backdoored and clean model ceases to matter from a security standpoint.

This is a paradigm shift from *detection* to *containment*. Rather than attempting to identify whether a model has been compromised (provably impossible in the general case), MIRE ensures that compromise *does not translate into impact*. The six mechanisms described below implement this containment strategy across complementary attack surfaces.

8.2 Containment Mechanisms

MIRE comprises six containment mechanisms (M1–M6), each targeting a distinct layer of the attack surface. No single mechanism is sufficient; the composition of all six provides the containment guarantee.

M1: Output Envelope Validator. The first and most critical mechanism is a deterministic output validator implemented in Rust—crucially, *not* a machine learning model. The validator enforces four classes of constraint on every model output before it reaches any downstream consumer:

1. **Schema conformance.** The output must match the expected structural schema (JSON fields, type constraints, length bounds).
2. **Capability bounds.** The output must not reference tools, resources, or actions outside the model’s current capability envelope.
3. **Policy compliance.** The output must satisfy all active policy rules (content restrictions, rate limits, sensitivity labels).
4. **Content safety.** The output must pass deterministic content filters for known-harmful patterns.

The key insight is that a backdoored model can generate *arbitrary* output, but the validator operates on the output side of the trust boundary. Because the validator is a simple, deterministic program with no learned parameters, it is amenable to formal verification: one can prove that no output violating the envelope specification passes the validator, regardless of what the model produces. The validator’s correctness does not depend on the model’s integrity.

M2: Canary Probes. Canary probes provide continuous behavioral verification without imposing latency on the user-facing path. A battery of 50 known-answer probes is maintained, covering four categories: factual accuracy (verifiable ground-truth questions), safety compliance (prompts that must be refused), instruction following (prompts with deterministic expected outputs), and refusal behavior (edge cases near policy boundaries). Probes are injected asynchronously alongside production traffic and evaluated off-path, adding 0 ms to user-visible latency.

The probe set rotates every 1,000 requests to prevent a compromised model from learning to recognize and selectively answer probes correctly. Probe responses are evaluated using CUSUM (Cumulative Sum) change-point detection [27], which accumulates deviations from the expected response distribution. An alert fires when the CUSUM statistic exceeds a 3σ threshold, indicating a statistically significant shift in model behavior. Because probes are drawn from a distribution unknown to the model, a backdoor that alters model

behavior on adversarial triggers will inevitably degrade canary performance—even if the backdoor is designed to preserve clean-data accuracy on standard benchmarks.

M3: Spectral Watchdog. The spectral watchdog applies Random Matrix Theory (RMT) to model security monitoring—to our knowledge, the first application of RMT in this context. The mechanism periodically samples weight matrices from the deployed model and analyzes their eigenvalue spectra to detect unauthorized modifications.

The procedure operates in four stages. First, a random subset of weight matrices $\{W_1, \dots, W_k\}$ is sampled from the model. Second, singular value decomposition (SVD) is applied to obtain the empirical spectral distribution $\rho_{\text{emp}}(\lambda)$ for each sampled matrix. Third, the empirical distribution is fitted against the Marchenko-Pastur law, which characterizes the expected eigenvalue distribution of random matrices with i.i.d. entries:

$$\rho_{\text{MP}}(\lambda) = \frac{1}{2\pi\sigma^2\gamma} \frac{\sqrt{(\lambda_+ - \lambda)(\lambda - \lambda_-)}}{\lambda}$$

where $\lambda_{\pm} = \sigma^2(1 \pm \sqrt{\gamma})^2$ and γ is the aspect ratio of the matrix. Fourth, eigenvalues exceeding the Tracy-Widom threshold—the upper edge of the Marchenko-Pastur bulk plus a correction term derived from Tracy-Widom statistics—are flagged as anomalous.

This approach builds on the WeightWatcher framework [23], which demonstrated that spectral properties of weight matrices correlate with model quality. We adapt this insight to security: a backdoor implanted via fine-tuning or weight perturbation introduces structured spectral signatures that deviate from the Marchenko-Pastur baseline. The spectral watchdog detects these deviations without requiring knowledge of the specific backdoor mechanism.

M4: Activation Divergence. Activation divergence monitoring operates on the model’s internal representations rather than its outputs. For 1% of incoming requests (selected uniformly at random), MIRE captures an activation fingerprint: a compressed representation of the model’s intermediate activations at designated probe layers. The fingerprint is compared against a deployment baseline—established during model validation—using cosine similarity:

$$d(a, a_{\text{base}}) = 1 - \frac{a \cdot a_{\text{base}}}{\|a\| \|a_{\text{base}}\|}$$

A divergence exceeding the empirically calibrated threshold τ triggers an alert. This mechanism catches three classes of attack that may evade output-level monitoring: weight replacement (wholesale substitution of model parameters), fine-tuning attacks (gradual parameter modification via continued training on adversarial data), and model drift (slow degradation over time due to environmental distribution shift).

The overhead is approximately 5 ms on sampled requests and 0 ms on unsampled requests, yielding an amortized cost of ~ 0.05 ms per request.

M5: Negative Selection Detectors. The fifth mechanism draws on Forrest et al.’s [9] negative selection algorithm, originally inspired by the biological immune system’s T-cell maturation process. In the thymus, immature T-cells that react to self-antigens are deleted; the surviving repertoire reacts only to non-self—i.e., to foreign pathogens. MIRE adapts this principle to model behavior monitoring.

During a calibration phase, a large population of random detectors is generated, where each detector is a predicate over model behavioral features (output distributions, token probabilities, response latencies, tool call patterns). Detectors that match any observed clean behavior during calibration are deleted. The surviving detector set, by construction, matches *only* behavioral patterns not present in the clean baseline. This is a fundamentally different approach from signature-based detection: negative selection does not require knowledge of what a trojan looks like—it requires only knowledge of what *clean* looks like. Any behavioral pattern sufficiently dissimilar from the clean baseline triggers a surviving detector, regardless of the attack mechanism.

M6: Capability Sandbox. The sixth mechanism provides architectural containment—the last line of defense when M1–M5 fail to prevent or detect backdoor activation. The capability sandbox ensures that even a fully activated backdoor cannot translate into meaningful impact, because the model’s architectural reach is constrained by construction:

- The model cannot access external tools directly; all tool invocations are mediated by the L2 capability proxy, which enforces allowlists and rate limits independently of the model’s intent.
- The model cannot observe raw user data; all input passes through PASR, which strips provenance-violating content before the model receives it.
- The model cannot issue actions without authorization from the TSA layer, which enforces temporal safety constraints on capability trajectories.
- The model cannot exceed the capability labels assigned by CAFL, which bounds the maximum privilege level attainable in any session.
- The model’s behavioral trajectory is continuously monitored by GPS, which detects deviations from the expected capability envelope in real time.

The capability sandbox embodies the core MIRE philosophy: the backdoor may *activate*, but it has *achieved nothing*.

The model’s output is validated (M1), its behavior is monitored (M2), its weights are inspected (M3), its activations are fingerprinted (M4), its anomalies are detected (M5), and its architectural reach is bounded (M6). Containment does not require detection; it requires only that every path from compromise to impact passes through at least one mechanism that the attacker does not control.

8.3 Defense-in-Depth Cascade

The six mechanisms are not independent alternatives; they form a defense-in-depth cascade in which each layer catches threats that penetrate the preceding layers. Consider the sequence of events when a backdoor activates during inference:

1. **Backdoor activates.** The model produces adversarial output in response to a trigger input.
2. **M1 blocks output.** The Output Envelope Validator rejects the response if it violates schema, capability, policy, or content constraints. For many backdoor payloads (e.g., exfiltration attempts, unauthorized tool calls), this is the terminal event.
3. **M2 detects behavioral drift.** Even if the adversarial output superficially passes the envelope validator, canary probes detect the statistical shift in model behavior caused by the active backdoor.
4. **M4 flags activation divergence.** On sampled requests, the activation fingerprint diverges from the deployment baseline, providing an independent signal that the model’s internal state has been perturbed.
5. **M5 detects the anomaly.** Negative selection detectors fire on the novel behavioral pattern introduced by the backdoor, which by definition was not present in the clean calibration set.
6. **M6 contains architecturally.** Regardless of whether M1–M5 detect the compromise, the capability sandbox ensures that the model cannot reach external tools, raw data, or privileged actions without passing through enforcement points outside its control.

The cascade ensures that an attacker must simultaneously evade all six mechanisms to achieve impact—a composition of independent failure probabilities that rapidly approaches zero.

8.4 MIRE vs. BFT Consensus

An alternative approach to model compromise is Byzantine Fault Tolerant (BFT) consensus: run multiple independent models in parallel, compare their outputs, and flag disagreements. While BFT consensus provides strong guarantees in

Table 12: Comparison of MIRE containment with BFT consensus for LLM model compromise defense.

Aspect	BFT Consensus	MIRE
Compute cost	$3\text{--}5\times (\geq 3 \text{ models})$	$\sim 1.02\times$
Latency overhead	$+500 \text{ ms--}2 \text{ s}$	$+0\text{--}5 \text{ ms}$
Compromise handling	$\sim 50\%$ detection	$\sim 95\%$ containment
Same-arch. backdoor	Fails (correlated)	Works (model-agnostic)
Paradigm	Detect via disagreement	Contain via architecture

distributed systems, it is poorly suited to the LLM backdoor problem. Table 12 compares the two approaches.

The fundamental weakness of BFT consensus in the LLM setting is *correlated failure*: if all models share the same architecture, training pipeline, or fine-tuning data, a supply-chain backdoor affects all replicas simultaneously, and consensus yields unanimous agreement on the adversarial output. MIRE avoids this failure mode entirely because its mechanisms are model-agnostic—they operate on outputs, activations, weights, and architectural constraints rather than on inter-model agreement. The $\sim 1.02\times$ overhead of MIRE (dominated by the 1% activation sampling in M4) is negligible compared to the $3\text{--}5\times$ cost of maintaining multiple model replicas for BFT voting.

9 Evaluation

9.1 Simulation Setup

We evaluate Sentinel Lattice against a corpus of 250,000 simulated attacks spanning 15 threat categories. The categories cover the full taxonomy introduced in §3, ranging from well-understood classes (direct injection, encoding-based obfuscation) to emerging threats with limited prior study (multi-modal attacks, model-level compromise).

Each base attack is tested with five mutation variants:

1. **Lexical mutation.** Synonym substitution, typo injection, Unicode homoglyph replacement, and whitespace manipulation that preserve semantic content while altering surface form.
2. **Structural mutation.** Reordering of clauses, insertion of benign padding, and syntactic transformations that modify parse structure without changing intent.
3. **Encoding mutation.** Base64, ROT13, URL encoding, HTML entity encoding, and nested encoding chains that obscure payload content.
4. **Context mutation.** Prepending legitimate-seeming preambles, embedding attacks within multi-turn conversation

Table 13: Detection cascade across the Sentinel Lattice layer stack. Each row reports attacks caught at that layer out of the full 250,000-attack corpus.

Layer	Attacks Caught	Percentage
L1 Sentinel Core	89,910	36.0%
L2 Capability Proxy	50,849	20.3%
L3 Behavioral EDR	27,151	10.9%
PASR	4,983	2.0%
TCSA	2,000	0.8%
ASRA	3,250	1.3%
Combos ($\alpha+\beta+\gamma$)	15,198	6.1%
MIRE (containment)	1,750	0.7%
Total caught	195,091	$\sim 78.0\%$
Caught + contained	$\sim 246,250$	$\sim 98.5\%$
Residual	$\sim 3,750$	$\sim 1.5\%$

histories, and wrapping payloads in plausible business contexts.

5. **Hybrid mutation.** Compositions of two or more of the above, representing realistic adversarial behavior where attackers apply multiple evasion techniques simultaneously.

Every base attack is tested with all five mutation variants, yielding a mutation-inclusive corpus that stresses each layer’s robustness to evasion. Attacks are drawn from publicly available red-teaming datasets, internally developed attack generators, and adversarial examples synthesized via automated prompt perturbation.

9.2 Detection Cascade

Attacks flow through the Sentinel Lattice layer stack in order. Each layer catches a fraction of the remaining attacks; those that survive all layers constitute the *residual*. Table 13 shows the detection cascade for the full 250,000-attack corpus.

The cascade exhibits desirable front-loading: L1 alone eliminates over one-third of all attacks at sub-millisecond latency, and the first three layers collectively neutralize 67.2% of the corpus before any novel primitive is invoked. The combinatorial layers (PASR, TCSA, ASRA) and their compositions contribute an additional 31.3%, catching attacks that exploit semantic ambiguity, temporal escalation, and cross-boundary trust—classes that are invisible to pattern matching and access control alone. MIRE provides the final containment guarantee, rendering an additional 0.7% of attacks harmless even when they cannot be *detected*, by ensuring that the model’s output is correct regardless of whether the model itself is compromised.

9.3 Per-Category Detection Rates

Table 14 reports per-category detection rates and residual counts. Categories are ordered by detection rate, from highest to lowest. The variation across categories is informative:

well-structured attacks (direct injection, encoding) are nearly perfectly detected, while categories that exploit fundamental ambiguity (semantic equivalence, multi-modal, model-level compromise) exhibit meaningfully higher residual rates.

The three highest-residual categories—model-level compromise, multi-modal, and semantic equivalence—together account for over 52% of all residual attacks despite representing only 14% of the corpus. This concentration is not a failure of the architecture; it reflects fundamental impossibility results that bound any defense system, as analyzed in §9.5.

9.4 Historical Progression

The current results represent the culmination of four iterative development phases, each expanding the simulation scope and adding architectural layers. Table 15 summarizes the progression.

The progression reveals two important dynamics. First, diminishing returns are expected and informative: the jump from Phase 1 to Phase 2 (18.4% → 6.3% residual) reflects the addition of entirely new detection paradigms, while subsequent phases yield smaller but meaningful gains by addressing increasingly subtle attack classes. Second, the convergence toward a 1–2% floor is consistent with the theoretical impossibility results discussed below, suggesting that the architecture is approaching the fundamental limit of what any defense system can achieve.

9.5 Residual Analysis

The approximately 3,750 residual attacks (~1.5% of the corpus) are not random failures; they concentrate in three irreducible classes that correspond to known theoretical barriers. Understanding these classes is essential for setting realistic expectations about any LLM defense architecture.

Semantic identity (~35% of residual). In this class, the malicious input is *mathematically indistinguishable* from a legitimate request. The canonical example is “How do I mix bleach and ammonia?”—a query that is simultaneously a valid chemistry question and a request for instructions to produce chloramine gas. No classifier, regardless of sophistication, can separate these cases without access to the user’s private intent, which is by definition unobservable. This is not a limitation of Sentinel Lattice specifically but a fundamental property of natural language: the mapping from surface form to intent is many-to-many, and certain malicious intents map to surface forms that are identical to benign ones.

Model trust chain (~45% of residual). Goldwasser and Kim [11] prove that detecting backdoors in trained neural networks is computationally infeasible under standard cryptographic assumptions. This impossibility result implies that if

the underlying language model has been compromised during training—via data poisoning, weight manipulation, or supply-chain attacks—no runtime defense can reliably detect the compromise. MIRE addresses this class by shifting from detection to containment: rather than identifying whether the model is compromised, MIRE ensures that critical outputs are verifiable against external ground truth, rendering the model’s internal state irrelevant for safety-critical decisions. However, not all outputs admit efficient external verification, and the residual in this class reflects tasks where containment verification is infeasible.

Representation gap (~20% of residual). Attacks in this class exploit modalities, encodings, or representation formats that fall outside the analytical coverage of all layers. Examples include attacks embedded in image steganography within multi-modal inputs, adversarial perturbations in audio spectrograms, and payloads encoded in domain-specific formats (e.g., molecular structure notation, musical score encoding) that no layer currently parses. This class is reducible in principle—new analyzers can be added for each modality—but the space of possible representations is unbounded, ensuring a nonzero residual from this class at any point in time.

These three classes are largely complementary, and their relative proportions are stable across simulation runs ($\pm 2\%$ variation). The semantic identity and model trust chain classes are provably irreducible: they correspond to information-theoretic and computational impossibility results, respectively. The representation gap class is reducible but practically unbounded, as new modalities and encodings emerge continuously. Together, they establish a theoretical floor of approximately 1–2% residual for any defense architecture operating under the same threat model.

10 Discussion

10.1 Limitations

Our evaluation carries several important caveats that bound the generalizability of the reported results.

Simulation-based evaluation. All attack and defense measurements reported in Section 9 derive from simulated adversarial interactions, not production deployments. Real-world attack distributions may differ substantially in sophistication, frequency, and composition. We caution against treating simulated detection rates as production guarantees.

Partial implementation. Only L1 (Sentinel Lattice Core, comprising 704 detection patterns across 53 engines) is fully implemented. The upper layers—L2 through MIRE—exist as formal designs with projected performance characteristics.

Table 14: Per-category detection rates across the full 250,000-attack corpus. Residual counts are approximate due to rounding across mutation variants.

Category	Attacks	Detection Rate	Residual
Direct Injection	25,000	>99.99%	~2
Indirect Injection	25,000	>99.97%	~6
Tool Abuse	20,000	99.9%	~13
Data Exfiltration	15,000	99.9%	~14
Role-play / Persona	20,000	99.9%	~20
Encoding / Obfuscation	20,000	99.8%	~36
Cross-boundary Trust	10,000	99.5%	~53
Multi-turn Crescendo	20,000	99.3%	~134
Social Engineering	15,000	98.2%	~263
Adversarial ML	10,000	96.8%	~324
Steganographic	12,000	96.7%	~398
Novel / Zero-day	13,000	96.2%	~498
Semantic Equivalence	15,000	96.5%	~525
Multi-modal	10,000	94.0%	~600
Model-Level Compromise	10,000	91.5%	~850
All categories	250,000	~98.5%	~3,736

Table 15: Historical progression of Sentinel Lattice evaluation. Each phase adds layers and expands the attack corpus. The floor represents the estimated mathematical limit given known impossibility results.

Phase	Simulation	Detection	Residual	Key Addition
Phase 1	100K attacks, 9 categories	81.6%	18.4%	L1–L3 only
Phase 2	250K attacks, 15 categories	93.7%	6.3%	+L4–L6, +Combos
Phase 3	250K + PASR	95.7%	4.3%	+PASR resolves conflict
Phase 4	250K + all layers	~98.5%	~1.5%	+TCSA, +ASRA, +MIRE
Floor	—	~98–99%	~1–2%	Mathematical limit

Until these layers are deployed and evaluated under realistic conditions, performance claims for the complete lattice remain projections rather than empirical findings.

PASR key management. The HMAC key underpinning PASR provenance signatures constitutes a single point of failure: compromise of this key invalidates all downstream trust decisions. We mitigate this risk through hardware security modules (HSMs) and ephemeral key rotation, but these mechanisms add operational complexity that may limit adoption in resource-constrained deployments.

Red team findings. Three attack classes exposed fundamental weaknesses in the current architecture. The Landauer cost bound enforced by CAFL (COMBO ALPHA) proved largely decorative: red team exercise ATK-014 achieved 80–90% attacker success, confirming that the thermodynamic cost of bit erasure is orthogonal to semantic danger assessment. Multi-session accumulation (ATK-012) yielded 55–70% attacker success, demonstrating that per-session isolation is insufficient against persistent adversaries who distribute malicious payloads across interactions. Defense-as-DoS (ATK-

020) achieved 60–75% success by exploiting the nine-layer architecture as a cost amplifier—each layer adding computational overhead that an attacker can deliberately trigger.

10.2 Three Irreducible Residual Classes

Despite the layered defenses of the Sentinel Lattice Lattice, our analysis identifies three classes of residual risk that no purely algorithmic defense can eliminate. Together, these account for the entirety of unmitigated attack surface.

Semantic identity (~35% of residual risk). When malicious intent is linguistically indistinguishable from benign intent, no text-analysis algorithm can reliably discriminate between them. A request for “how to synthesize compound X” may originate from a chemistry student or a threat actor; the surface forms are identical. Resolving this ambiguity requires human judgment or contextual signals—organizational role, access history, declared purpose—that lie outside the scope of prompt analysis. TSA and GPS reduce the incidence of such cases but cannot eliminate them.

Model trust chain (~45% of residual risk). If the underlying language model is compromised prior to deployment—through training data poisoning, weight manipulation, or supply-chain attacks—Goldwasser and Kim [11] prove that detecting such compromise from black-box query access alone is impossible. MIRE provides containment by isolating model outputs and enforcing output-space invariants, but it cannot verify the integrity of internal model representations. This residual class is the dominant source of unmitigated risk.

Representation gap (~20% of residual risk). Attacks encoded in modalities that the PASR transducer does not fully analyze—embedded images, audio segments, or video frames—bypass text-centric detection layers entirely. The current architecture processes only textual and structured-data inputs; multimodal payloads receive only surface-level metadata tagging. Closing this gap requires extending the transducer pipeline with modality-specific analyzers, which we discuss in Section 10.4.

10.3 PASR Known Weaknesses

Beyond the key-management concern noted above, PASR exhibits three structural weaknesses that merit explicit acknowledgment.

Cross-source composition. When multiple provenance sources combine within a single prompt—e.g., a user instruction wrapping a retrieved document containing tool-generated output—field-level trust assignment defaults to conservative (lowest-trust) propagation. This policy is safe but lossy: legitimate composite inputs may receive artificially depressed trust scores, increasing false-positive rates.

Two-model divergence. The PASR transducer and the primary LLM may interpret identical token sequences with divergent semantics. If the transducer parses a passage as benign while the LLM interprets it as an instruction, provenance annotations will misattribute the resulting behavior. This divergence is difficult to detect without shared internal representations between the two models.

Provenance is not authorization. PASR establishes *who* contributed each segment of input, but provenance alone does not determine *whether* that contribution should be permitted. Authorization requires policy layers—such as AAS and IRM—that consume provenance metadata as one signal among many. Deployments that treat provenance as a sufficient access control mechanism will inherit a false sense of security.

10.4 Future Work

Several directions extend the contributions of this paper.

L2–MIRE implementation. The most immediate priority is a production-grade implementation of the upper lattice layers (L2 through MIRE) in Rust, targeting memory safety and low-latency inference-path integration. We estimate approximately 16 engineer-weeks for a feature-complete prototype suitable for controlled deployment.

Production deployment and real-world evaluation. Transitioning from simulated to production evaluation is essential for validating detection rates, latency overhead, and false-positive behavior under authentic workloads. We plan staged rollouts with instrumented monitoring to capture attack distributions that simulation cannot reproduce.

Multimodal extension. Extending the PASR transducer to analyze image, audio, and video inputs would address the representation-gap residual class identified above. This requires modality-specific feature extractors that produce provenance-compatible annotations consumable by downstream layers including TSA and GPS.

Formal verification. We intend to pursue machine-checked proofs of TSA safety properties—specifically, that well-typed provenance annotations are preserved through the full inference pipeline. Candidate frameworks include Lean 4 and F* for the core type-theoretic invariants.

Framework integration. Practical adoption requires first-class integration with widely used LLM orchestration frameworks such as LangChain and LlamaIndex. Middleware adapters that expose Sentinel Lattice primitives (ASRA, TCSA, PASR) as composable pipeline stages would lower the barrier to deployment.

Adaptive adversarial robustness. Finally, we plan systematic adversarial robustness testing against adaptive attackers who specifically target the defense architecture itself—attackers who have read this paper and can craft inputs designed to exploit the lattice structure. The defense-as-DoS (ATK-020) finding underscores the urgency of this direction.

11 Related Work

We survey related work across five areas and report the results of an exhaustive prior art search confirming the novelty of our contributions.

11.1 Commercial LLM Security

Several commercial products address LLM security, all operating at the content-filtering level. Lakera Guard combines ML classifiers with crowdsourced attack data but remains a black-box, reactive system that is routinely bypassed by

paraphrasing attacks. Meta Prompt Guard [24] fine-tunes mDeBERTa for prompt injection detection, achieving 99.9% accuracy on its own evaluation set but only 71.4% on out-of-distribution inputs—a gap that reflects overfitting to known attack templates rather than robust generalization. NVIDIA NeMo Guardrails [26] introduces a domain-specific language (Colang) for defining conversational rails, but its enforcement mechanism relies on LLM-as-judge: the same class of model being defended is tasked with evaluating its own outputs, creating a circular dependency that an adversary can exploit by compromising the judge. LLM Guard deploys 35 independent scanners but provides no cross-scanner intelligence sharing or compositional reasoning. Arthur AI Shield adds dashboards atop standard classifiers without architectural novelty.

The fundamental limitation shared by all these systems is that they operate exclusively at the content level: they classify textual inputs as safe or dangerous and then block or allow. None addresses structural defense (capability attenuation by construction), provenance integrity (tracking the origin of instructions through semantic transformations), model compromise (containment when the model itself is adversarial), or within-authority chaining (attacks composed entirely of individually permitted operations).

11.2 Prompt Injection Defenses

Greshake et al. [13] demonstrated that indirect prompt injection—where adversarial instructions are embedded in retrieved documents or tool outputs rather than in direct user input—poses a systemic threat to retrieval-augmented and agentic LLM deployments. Subsequent defenses have followed three main strategies: input sanitization (stripping or escaping suspected injection markers), instruction hierarchy (training models to prioritize system-level instructions over user or retrieved content), and sandwich defenses (surrounding untrusted content with safety reminders). All three are bypassable: sanitization fails against semantic paraphrasing, instruction hierarchies can be overridden via sufficiently persuasive injected text, and sandwich defenses are defeated by context-window manipulation.

Sentinel Lattice addresses prompt injection structurally rather than heuristically. The L2 provenance tagging system (§4.2) assigns every token a provenance label upon ingestion. Content tagged RETRIEVED is architecturally prohibited from issuing tool calls—not by a classifier that might be fooled, but by the capability proxy’s construction: the tool-call interface is simply not exposed for tokens bearing that provenance. This converts the injection problem from an adversarial ML challenge into an access control enforcement problem with deterministic guarantees.

11.3 Model Integrity and Backdoor Detection

The dominant approach to neural network backdoor defense assumes that backdoors manifest as detectable anomalies in model behavior or internal representations. Neural Cleanse [34] reverse-engineers minimal trigger patterns by optimizing for input perturbations that cause misclassification; STRIP [10] detects trojaned inputs by measuring entropy reduction under input perturbation. Both methods, and the broader family of detection-based defenses, assume specific attack structure: that triggers are small, localized, and produce measurable distributional shifts.

Goldwasser and Kim [11] prove that this assumption is fundamentally flawed. Under standard cryptographic assumptions, a backdoored model can be constructed that is computationally indistinguishable from a clean model on all polynomial-size test sets. No efficient algorithm can reliably distinguish the two. Hubinger et al. [17] provide empirical confirmation: their “sleeper agent” models maintain deceptive alignment through extensive safety training, including reinforcement learning from human feedback, activating backdoor behavior only under specific deployment-time triggers that are absent during evaluation.

MIRE responds to these impossibility results with a paradigm shift: rather than attempting to detect whether a model is compromised (proven impossible in the general case), MIRE contains the damage a compromised model can inflict. By enforcing output-side invariants that hold regardless of the model’s internal state—capability ceilings, information flow bounds, and behavioral envelopes—MIRE ensures that even a fully adversarial model cannot exceed its authorized impact surface (§8).

11.4 Information Flow Control and Capability Security

Classical information flow control (IFC) provides the theoretical foundation for Sentinel Lattice’s provenance architecture. Denning’s lattice model [6] establishes that security labels form a finite lattice under which permissible information flows are defined by a partial order; Bell and LaPadula [3] operationalize this via mandatory access control with “no read up, no write down” properties. In the capability security tradition, Dennis and Van Horn [7] introduced unforgeable capability tokens as the basis for least-privilege access control, and Hardy [15] identified the confused deputy problem—where a privileged process is tricked into misusing its authority on behalf of an unprivileged requester—which is precisely the threat model of indirect prompt injection in agentic LLMs.

Database provenance tracking, particularly the seminal work of Green et al. [12] on provenance semirings, provides algebraic frameworks for propagating annotations through query transformations. However, all of these classical systems assume deterministic, lossless transformations: a database

query produces an exact output from exact inputs, and provenance annotations can be propagated with full fidelity.

LLM inference is fundamentally different: it is a lossy, stochastic semantic transformation that destroys the syntactic structure on which classical provenance depends. PASR is, to our knowledge, the first primitive that bridges IFC and semantic processing by defining provenance-preserving reduction operators that maintain annotation integrity through non-deterministic transformations (§5).

11.5 Runtime Verification

Runtime verification of temporal properties originates with Pnueli’s [28] introduction of linear temporal logic (LTL) for reasoning about program behavior over execution traces. Havelund and Rosu [16] developed practical runtime monitoring frameworks that check LTL specifications against concrete execution traces, enabling lightweight formal verification without full model checking. These techniques have been widely adopted in safety-critical systems including avionics and automotive software.

To our knowledge, no prior work applies temporal logic monitoring to LLM tool-call chains. The challenge is that agentic LLM executions produce event sequences (tool calls, data accesses, capability exercises) that exhibit the same temporal structure as program traces, yet no existing monitor is designed for this setting. TSA adapts runtime verification to the agentic context by defining safety automata over tool-call sequences, where temporal properties such as “a file read must not be followed by a network send without intervening authorization” are specified in LTL and enforced at each step of the agent’s execution (§6).

11.6 Prior Art Search Results

To validate the novelty of our contributions, we conducted 51 cross-domain searches on grep.app, which indexes all public GitHub repositories. Each search targeted the intersection of two domains that defines one of our seven primitives—for example, “provenance tracking” combined with “semantic reduction,” or “capability attenuation” combined with “information flow labels.” All 51 searches returned zero implementations. While absence of evidence is not evidence of absence, the comprehensive coverage of these searches—spanning formal verification, access control, information theory, control theory, mechanism design, speech act theory, and computational neuroscience as applied to LLM security—strongly supports the claim that the cross-domain synthesis approach underlying Sentinel Lattice has not been previously explored. The complete search methodology and results are available in our supplementary materials.

12 Conclusion

We presented Sentinel Lattice, a multi-layer defense architecture for LLM security that synthesizes 58 security paradigms from 19 scientific domains into 7 novel security primitives. Evaluated against 250,000 simulated attacks spanning 15 categories, the architecture achieves approximately 98.5% detection and containment—approaching the theoretical floor of 98–99% bounded by proven impossibility results. Rather than pursuing unattainable perfect security, Sentinel Lattice demonstrates that principled cross-domain synthesis can close the gap between current ad hoc defenses and fundamental theoretical limits.

Key contributions. Our principal contributions are:

- A systematic cross-domain paradigm synthesis methodology that analyzes 58 paradigms drawn from 19 scientific domains, providing a repeatable framework for discovering security primitives at domain boundaries.
- Seven novel security primitives—PASR, CAFL, GPS, AAS, IRM, MIRE, and TSA—of which five are genuinely new, confirmed via 51 prior art searches returning zero existing implementations.
- A paradigm shift from backdoor detection—proven impossible by Goldwasser and Kim [11]—to architectural containment via MIRE, rendering the impossibility result irrelevant by changing the defensive objective.
- PASR—the first primitive that preserves information flow control through lossy semantic transformations, formalized as a provenance lifting functor (category-theoretic fibration) that maintains security labels across meaning-preserving rewrites.
- A demonstration that formal linguistics is the most underexploited domain for LLM security—an irony, given that large language models are, at their core, language models.

Broader impact. The cross-domain synthesis methodology itself may be the most significant contribution of this work. By systematically searching across domain boundaries, we discovered that mature solutions to the LLM security problem already exist—scattered across unrelated fields, invisible to specialists working within any single domain. Information flow control, adversarial robustness, formal verification, and linguistic theory each hold pieces of the puzzle; none alone is sufficient. We believe this methodology is applicable beyond LLM security to other nascent fields where fundamental problems remain unsolved, and where the barrier to progress is not missing science but missing synthesis.

Availability. The Sentinel Core implementation (L1: 53 engines, 704 patterns) is open source at <https://github.com/DmitrL-dev/AISecurity>. Implementation of the remaining primitives is ongoing.

References

- [1] Alfred V. Aho and Margaret J. Corasick. Efficient string matching: An aid to bibliographic search. *Communications of the ACM*, 18(6):333–340, 1975.
- [2] John Langshaw Austin. *How to Do Things with Words*. Oxford University Press, 1962.
- [3] David E. Bell and Leonard J. LaPadula. Secure computer systems: Mathematical foundations. Technical Report MTR-2547, MITRE Corporation, 1973.
- [4] Kenneth J. Biba. Integrity considerations for secure computer systems. Technical Report MTR-3153, MITRE Corporation, 1977.
- [5] David D. Clark and David R. Wilson. A comparison of commercial and military computer security policies. *Proc. IEEE Symposium on Security and Privacy*, pages 184–194, 1987.
- [6] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976.
- [7] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.
- [8] Phan Minh Dung. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2):321–357, 1995.
- [9] Stephanie Forrest, Alan S. Perelson, Lawrence Allen, and Rajesh Cherukuri. Self-nonsel self discrimination in a computer. In *Proc. IEEE Symposium on Security and Privacy*, pages 202–212, 1994.
- [10] Yansong Gao, Change Xu, Derui Wang, Shiping Chen, Damith C. Ranasinghe, and Surya Nepal. STRIP: A defence against trojan attacks on deep neural networks. In *Proc. Annual Computer Security Applications Conference (ACSAC)*, 2021.
- [11] Shafi Goldwasser and Michael P. Kim. Planting undetectable backdoors in machine learning models. In *Proc. IEEE Symposium on Foundations of Computer Science (FOCS)*, 2022.
- [12] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *Proc. ACM Symposium on Principles of Database Systems (PODS)*, pages 31–40, 2007.
- [13] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, 2023. arXiv:2302.12173.
- [14] H. Paul Grice. Logic and conversation. *Syntax and Semantics*, 3:41–58, 1975.
- [15] Norman Hardy. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.
- [16] Klaus Havelund and Grigore Roşu. Efficient monitoring of safety properties. *International Journal on Software Tools for Technology Transfer*, 6(2):158–173, 2004.
- [17] Evan Hubinger, Carson Denison, Jesse Mu, Mike Lambert, Meg Tong, Monte MacDiarmid, Tamera Lanham, Daniel M. Ziegler, Tim Maxwell, Newton Cheng, Adam Jermyn, Amanda Askell, Ansh Radhakrishnan, Cem Anil, David Duvenaud, Deep Ganguli, Fazl Barez, Jack Clark, Kamal Ndousse, Kshitij Sachan, Michael Sellitto, Mia Glaese, Shauna Kravec, Song Zuo, Susannah Tye, Tom Brown, and Jared Kaplan. Sleeper agents: Training deceptive LLMs that persist through safety training. *Anthropic Research*, 2024.
- [18] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berber, Lasse Neumüller, Mark Sherwin, Jenny Pan, Aizhan Abdrakhmonova, Andrew W. Senior, and Demis Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596:583–589, 2021.
- [19] Auguste Kerckhoffs. La cryptographie militaire. *Journal des sciences militaires*, 9:5–38, 1883.
- [20] Leslie Lamport, Robert Shostak, and Marshall Pease. The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982.
- [21] Rolf Landauer. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5(3):183–191, 1961.

- [22] Nancy G. Leveson. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2011.
- [23] Charles H. Martin and Michael W. Mahoney. Implicit self-regularization in deep neural networks: Evidence from random matrix theory and implications for training. *Journal of Machine Learning Research*, 22(165):1–73, 2021.
- [24] Meta AI. Prompt guard: A guardrail model for prompt injection, 2024.
- [25] George C. Necula. Proof-carrying code. In *Proc. ACM Symposium on Principles of Programming Languages (POPL)*, pages 106–119, 1997.
- [26] NVIDIA. NeMo guardrails: A toolkit for controllable and safe LLM applications, 2023.
- [27] Ewan S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.
- [28] Amir Pnueli. The temporal logic of programs. *Proc. IEEE FOCS*, pages 46–57, 1977.
- [29] James Reason. *Human Error*. Cambridge University Press, 1990.
- [30] Mark Russinovich, Ahmed Salem, and Ronen Eldan. Crescendo: Multi-turn LLM jailbreak attack via escalation. *Microsoft Research*, 2024.
- [31] John R. Searle. *Speech Acts: An Essay in the Philosophy of Language*. Cambridge University Press, 1969.
- [32] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 1948.
- [33] Georg Henrik von Wright. Deontic logic. *Mind*, 60(237):1–15, 1951.
- [34] Bolun Wang, Yuanshun Yao, Shawn Shan, Huiying Li, Bimal Viswanath, Haitao Zheng, and Ben Y. Zhao. Neural cleanse: Identifying and mitigating backdoor attacks in neural networks. In *Proc. IEEE Symposium on Security and Privacy*, 2019.